The Journal of Open Source Software

PyCFRL: A Python library for counterfactually fair offline reinforcement learning via sequential data preprocessing

Jianhan Zhang [1], **Jitao Wang**[2], **Chengchun Shi**[3], **John D. Piette**[4], **Donglin Zeng**[2], **and Zhenke Wu** [2¶]

1 Department of Statistics, University of Michigan, USA **2** Department of Biostatistics, University of Michigan, USA **3** Department of Statistics, London School of Economics, UK **4** Department of Health Behavior and Health Equity, School of Public Health, University of Michigan, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Reinforcement learning (RL) aims to learn and evaluate a sequential decision rule, often referred to as a "policy", that maximizes expected discounted cumulative rewards to optimize the population-level benefit in an environment across possibly infinitely many time steps. RL has gained popularity in fields such as healthcare, banking, autonomous driving, and, more recently, large language model fine-tuning. However, the sequential decisions made by an RL algorithm, while optimized to maximize overall population benefits, may disadvantage certain individuals who are in minority or socioeconomically disadvantaged groups. A fairness-unaware RL algorithm learns an optimal policy that makes decisions based on the *observed* state variables. However, if certain values of the sensitive attribute influence the state variables and lead the policy to systematically withhold certain actions from an individual, unfairness will result. For example, Hispanics may under-report their pain levels due to cultural factors, misleading a fairness-unaware RL agent to assign less therapist time to these individuals (Piette et al., 2023). Deployment of RL algorithms without careful fairness considerations can raise concerns and erode public trust in high-stakes settings.

To formally define and address the fairness problem in the novel sequential decision-making settings, Wang et al. (2025) extended the concept of single-stage counterfactual fairness (CF) in a structural causal framework (Kusner et al., 2017) to the multi-stage setting and proposed a data preprocessing algorithm that ensures CF. A policy is counterfactually fair if, at every time step, the probability of assigning any action does not change had the individual's sensitive attribute taken a different value, while holding constant other historical exogenous variables and actions. In this light, the data preprocessing algorithm ensures CF by constructing new state variables that are not impacted by the sensitive attribute(s). Reward preprocessing is also conducted, but with a different purpose to improve the value of the learned optimal policy rather than to ensure CF. We refer interested readers to Wang et al. (2025) for more technical details.

The `PyCFRL` library implements the data preprocessing algorithm proposed by Wang et al. (2025) and provides functionalities to evaluate the value (expected discounted cumulative reward) and counterfactual unfairness level achieved by any given policy. Here, "CFRL" stands for "Counterfactual Fairness in Reinforcement Learning". The library produces preprocessed trajectories that can be used by an off-the-shelf offline RL algorithm, such as fitted Q-iteration (FQI) (Riedmiller, 2005), to learn an optimal CF policy. The library can also simply read in any policy following a required format and return its value and counterfactual unfairness level in the environment of interest, where the environment can be either pre-specified or learned

DRAFT

from the data.

Statement of Need

Many existing Python libraries implement algorithms designed to ensure fairness in machine learning. For example, `Fairlearn` (Weerts et al., 2023) and `aif360` (Bellamy et al., 2018) provide tools for mitigating bias in single-stage machine learning predictions under statistical association-based fairness criteria such as demographic parity and equal opportunity. However, existing libraries do not focus on counterfactual fairness, which defines an individual-level fairness concept from a causal perspective, and they cannot be easily extended to the general RL setting. Scripts available from `ml-fairness-gym` (D'Amour et al., 2020) allow users to simulate unfairness in sequential decision-making, but they neither implement algorithms that reduce unfairness nor address CF. To our knowledge, Wang et al. (2025) is the first work to study CF in RL. Correspondingly, PyCFRL is also the first code library to address CF in the RL setting.

The contribution of PyCFRL is two-fold. First, PyCFRL implements a data preprocessing algorithm that ensures CF in offline RL. For each individual in the data, the preprocessing algorithm sequentially estimates and concatenates the counterfactual states under different sensitive attribute values with the observed state at each time point into a new state vector. The preprocessed data can then be directly used by existing RL algorithms for policy learning, and the learned policy will be counterfactually fair up to finite-sample estimation accuracy. Second, PyCFRL provides a platform for assessing RL policies based on CF. After passing in any policy and a data trajectory from the environment of interest, users can estimate the value and counterfactual unfairness level achieved by the policy in the environment of interest.

High-level Design

The PyCFRL library is composed of 5 major modules as summarized below.

Module	Functionalities
`reader`	Implements functions that read tabular trajectory data into an array format required by PyCFRL. Also implements functions that export trajectory data to the tabular format.
`preprocessor`	Implements the data preprocessing algorithm introduced in Wang et al. (2025).
`agents`	Implements an FQI algorithm (Riedmiller, 2005), which learns RL policies and makes decisions based on the learned policy.
`environment`	Implements a synthetic environment that produces synthetic data as well as a simulated environment that estimates and simulates the transition dynamics of the unknown environment underlying some real-world RL trajectory data. Also implements functions for sampling trajectories from the synthetic and simulated environments.
`evaluation`	Implements functions that evaluate the value and counterfactual unfairness level of a policy.

DRAFT

The Journal of Open Source Software

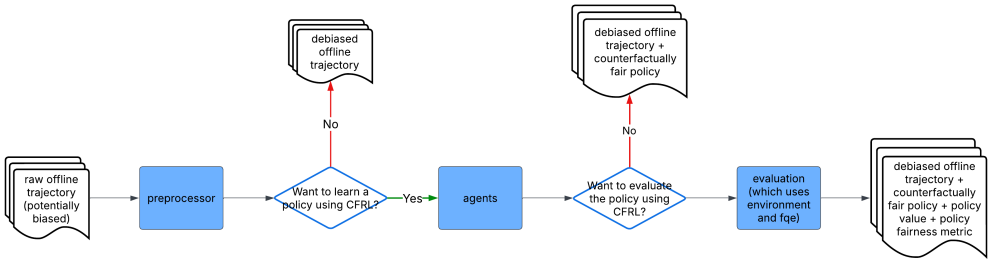


Figure 1: PyCFRL Workflow Chart

A general PyCFRL workflow is summarized in the diagram above: First, simulate trajectories using `environment` or read in trajectories using `reader`. Then, train a preprocessor using `preprocessor` and preprocess the training trajectory data. After that, pass the preprocessed trajectories into the FQI algorithm in `agents` to learn a counterfactually fair policy. Finally, use functions in `evaluation` to evaluate the value and counterfactual unfairness level of the trained policy.

In addition, PyCFRL also provides tools to check for potential non-convergence that may arise during the training of neural networks, FQI, or fitted-Q evaluation (FQE). More discussions about non-convergence in PyCFRL can be found in the "Common Issues" section of the documentation.

Data Examples

In the "Example Workflows" section of the documentation, we provide data examples with code to demonstrate some major workflows of PyCFRL. We also record the computing times of different workflows under different combinations of the number of individuals (N) and the length of horizons (T) in the "Computing Times" section of the documentation.

Conclusions

PyCFRL is a Python library that enables counterfactually fair reinforcement learning through data preprocessing. It also provides tools to calculate the value and unfairness level of a given policy. To our knowledge, it is the first library to address CF problems in the context of RL. The practical utility of PyCFRL can be further improved via extensions. First, the current PyCFRL implementation requires every individual in the offline dataset to have the same number of time steps. Extending the library to accommodate variable-length episodes can improve its flexibility and usefulness. Second, PyCFRL can further combine the preprocessor with popular offline RL algorithm libraries such as `d3rlpy` (Seno & Imai, 2022), or connect the evaluation functions with established RL environment libraries such as `gym` (Towers et al., 2024). Third, generalization to non-additive counterfactual states reconstruction can make PyCFRL more versatile. We leave these extensions to future updates.

Acknowledgements

Jianhan Zhang and Jitao Wang contributed equally to this work. The authors declare no conflicts of interest.

DRAFT

References

Bellamy, R. K. E., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Lohia, P., Martino, J., Mehta, S., Mojsilovic, A., Nagar, S., Ramamurthy, K. N., Richards, J., Saha, D., Sattigeri, P., Singh, M., Varshney, K. R., & Zhang, Y. (2018). *AI Fairness 360: An extensible toolkit for detecting, understanding, and mitigating unwanted algorithmic bias*. https://doi.org/10.48550/arXiv.1810.01943

D'Amour, A., Srinivasan, H., Atwood, J., Baljekar, P., Sculley, D., & Halpern, Y. (2020). Fairness is not static: Deeper understanding of long term fairness via simulation studies. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 525–534. https://doi.org/10.1145/3351095.3372878

Kusner, M., Loftus, J., Russell, C., & Silva, R. (2017). Counterfactual fairness. *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 4069–4079. ISBN: 9781510860964

Piette, J. D., Thomas, L., Newman, S., Marinec, N., Krauss, J., Chen, J., Wu, Z., & Bohnert, A. S. B. (2023). An automatically adaptive digital health intervention to decrease opioid-related risk while conserving counselor time: Quantitative analysis of treatment decisions based on artificial intelligence and patient-reported risk measures. *J Med Internet Res*, *25*, e44165. https://doi.org/10.2196/44165

Riedmiller, M. (2005). Neural fitted Q iteration – first experiences with a data efficient neural reinforcement learning method. In J. Gama, R. Camacho, P. B. Brazdil, A. M. Jorge, & L. Torgo (Eds.), *Machine learning: ECML 2005* (pp. 317–328). Springer Berlin Heidelberg. https://doi.org/10.1007/11564096_32

Seno, T., & Imai, M. (2022). d3rlpy: An offline deep reinforcement learning library. *Journal of Machine Learning Research*, *23*(315), 1–20. http://jmlr.org/papers/v23/22-0017.html

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., & others. (2024). Gymnasium: A standard interface for reinforcement learning environments. *arXiv Preprint arXiv:2407.17032*. https://doi.org/10.48550/arXiv.2407.17032

Wang, J., Shi, C., Piette, J. D., Loftus, J. R., Zeng, D., & Wu, Z. (2025). *Counterfactually fair reinforcement learning via sequential data preprocessing*. https://doi.org/10.48550/arXiv.2501.06366

Weerts, H., Dudík, M., Edgar, R., Jalali, A., Lutz, R., & Madaio, M. (2023). Fairlearn: Assessing and improving fairness of AI systems. In *Journal of Machine Learning Research* (No. 257; Vol. 24, pp. 1–8). http://jmlr.org/papers/v24/23-0389.html

DRAFT